BrainAI

ProSuite Industrial

CLI & AUTOMATION

PRODUCT DOCUMENTATION

BrainAI (Inhaber: Sascha Alexander Köhne)

BrainAI Inhaber: Sascha Alexander Köhne	PROSUITE INDUSTRIAL	🟢 PUBLIC CLASSIFICATION
Core Architecture & Development	**ProKey Command Line Reference**	Access Level: Public / Beta

---

Revision History

Version	Date	Author	Status	Description
1.0.0	2025-01-01	S. Köhne	Stable	Initial Hardware Entropy CLI
1.1.0	2026-02-27	S. Köhne	Stable	Hex-Output Support, Strict Length Validation & Pipeline Optimization

1 Product Overview

The **ProKey Executable** (`prokey.exe` / `prokey`) is the high-performance Command-Line Interface (CLI) for generating true hardware entropy. Unlike software-based pseudo-random number generators (PRNGs), ProKey delivers absolute random values, making it ideal for generating cryptographic keys (AES, RSA, ECC) or highly secure session tokens.

Core Features of the CLI:

- **Flexible Key Lengths:** Native support for the industrial standards of 128, 256, 512, and 1024 bits.
- **Raw & Hex Formats:** Keys can be output either as memory-efficient raw binary data or as human-readable, transmission-ready hex strings.
- **Pipeline Architecture:** Full support for `stdout` piping (`-O -`), allowing keys to be seamlessly passed to other crypto tools (such as OpenSSL or GPG) in Unix/Windows environments.
- **Zero-Trust Memory Wipe:** After the data is generated and written, the application's reserved RAM is rigorously overwritten with zeros to prevent cold boot attacks.

2 Technical Specifications

2.1 Command-Line Parameters (CLI)

The executable expects the following syntax: `prokey [OPTIONS]`

Parameter	Short	Description	Default Value
`--output <file>`	`-O`	Path to the output file. If `-` is specified, it writes directly to `stdout`.	`default.kedc`
`--length <bits>`	`-L`	Key length in bits. Allowed values: `128`, `256`, `512`, `1024`.	`256`

`--hex`	`-H`	Converts the binary entropy into a hexadecimal string before outputting it.	*Disabled (Raw)*
`--help`	`-h`	Displays the help menu and exits the program.	-

3 Global Error Management (Exit Codes)

To ensure seamless integration into scripts, the ProKey executable uses standardized exit codes. All status messages and errors are strictly output via `stderr` so they do not corrupt crypto pipelines in `stdout`.

Exit Code	***Error Source***	***Description & Resolution***
0	**SUCCESS**	The key was successfully generated and written.
1	**ERR_PARAM**	Unsupported key length requested. Correct the value to 128, 256, 512, or 1024.
1	**ERR_MEM**	RAM allocation failed (very rare, occurs only during critical memory shortage).
1	**ERR_IO**	File could not be opened/written (e.g., missing write permissions in the target directory).

4 Linux / Bash Integration

On Linux, ProKey is the ideal tool for securely initializing crypto containers (e.g., LUKS) or supplying automation processes with true entropy.

4.1 Example 1: Creating an AES-256 Master Key (Raw)

```bash
#!/bin/bash
KEY_FILE="/etc/proed/master_key.bin"

echo "Generating hardware key..."
./prokey -L 256 -O "$KEY_FILE"

if [ $? -eq 0 ]; then
    chmod 400 "$KEY_FILE" # Only root may read
    echo "Key successfully saved."
else
    echo "Critical error during key generation!"
    exit 1
fi
```

4.2 Example 2: Pipeline Generation for OpenSSL

Instead of writing the key to disk first (which can be a security risk), we generate it on-the-fly and pass it as a hex string directly to another process:

```bash
# Uses ProKey as a Hex RNG source for crypto processes
SECRET=$(./prokey -L 512 -H -O -)
echo "Your temporary session token: $SECRET"
```

5 Windows PowerShell Integration

On Windows systems, the CLI is immunized against a known pipeline issue (`\n` to `\r\n` conversion in `stdout`) by explicitly operating the stream in binary mode (`_O_BINARY`).

5.1 Example: Generating an API Token in Hex Format

```
Write-Host "Initializing hardware entropy..."

# Generate a 1024-bit key as a hex file
$process = Start-Process -FilePath "prokey.exe" -ArgumentList "-L 102

if ($process.ExitCode -eq 0) {
    Write-Host "API Token successfully created." -ForegroundColor Gre
} else {
    Write-Host "Hardware error during key generation (Code: $($proces
}
```

6 Best Practices & Security Guidelines

- **Raw vs. Hex:** Use the raw format (default) when the key is saved locally to a disk as a file (`.bin` / `.key`) for machine use. Use the `-H` (`--hex`) parameter when the key must be transmitted over text channels (JSON, REST APIs, configuration files) or typed out by humans.
- **Pipeline Security:** Piping directly into `stdout` (`-O -`) bypasses the hard drive completely. This is the most secure method to inject keys into RAM disks or directly into running applications.
- **No Console Pollution:** All status messages, such as `[+] Generated 256-Bit Key`, are always written to `stderr` by the engine. This means you can safely use the `> key.txt` redirect without the actual key being corrupted by text messages.

© 2026 Sascha Köhne	✉️ **Contact: koehne83@googlemail.com**	🛡️ **ProTU-Verified**
CEO / System Architect	BrainAI Inhaber : Sascha Alexander Köhne	We don't need **BRUTEFORCE**, we know **PHYSICS**